



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 36

Dr. Asad Mahmood

April 28, 2026

Lecture Outline

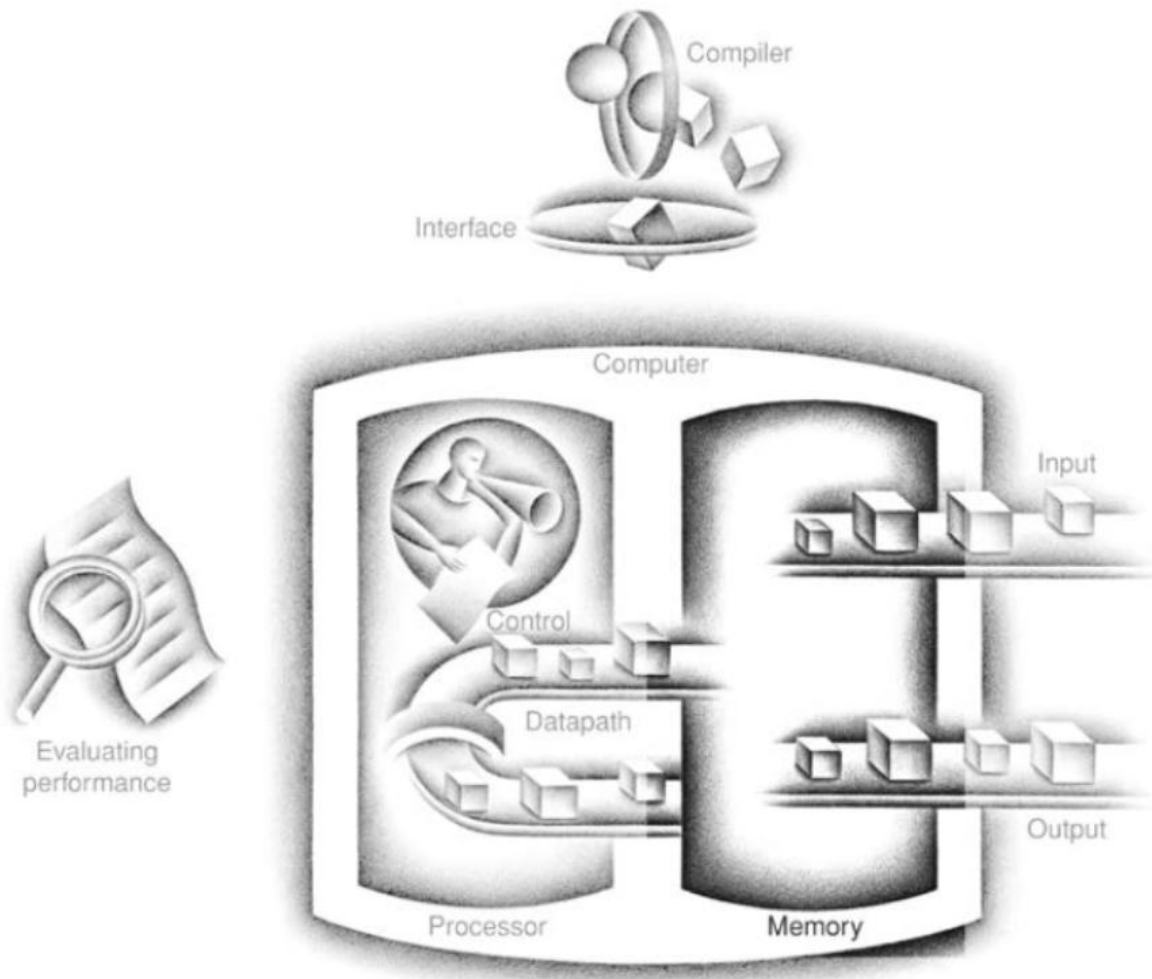
- Announcements (Assignment and Quiz 4)
- Outline of Previous Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.1 Introduction
 - 5.2 Memory Technologies
- Outline of Today's Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.2 Memory Technologies
 - 5.3 The Basics of Caches

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Introduction

Ideally one would desire an indefinitely large memory capacity such that any particular ... word would be immediately available. ...

We are ... forced to recognize the possibility of constructing a *hierarchy of memories*, each of which has greater capacity than the preceding but which is less quickly accessible

A. W. Burks, H. H. Goldstine, and J. von Neumann,
*Preliminary Discussion of the Logical Design of an
Electronic Computing Instrument*, 1946

Introduction

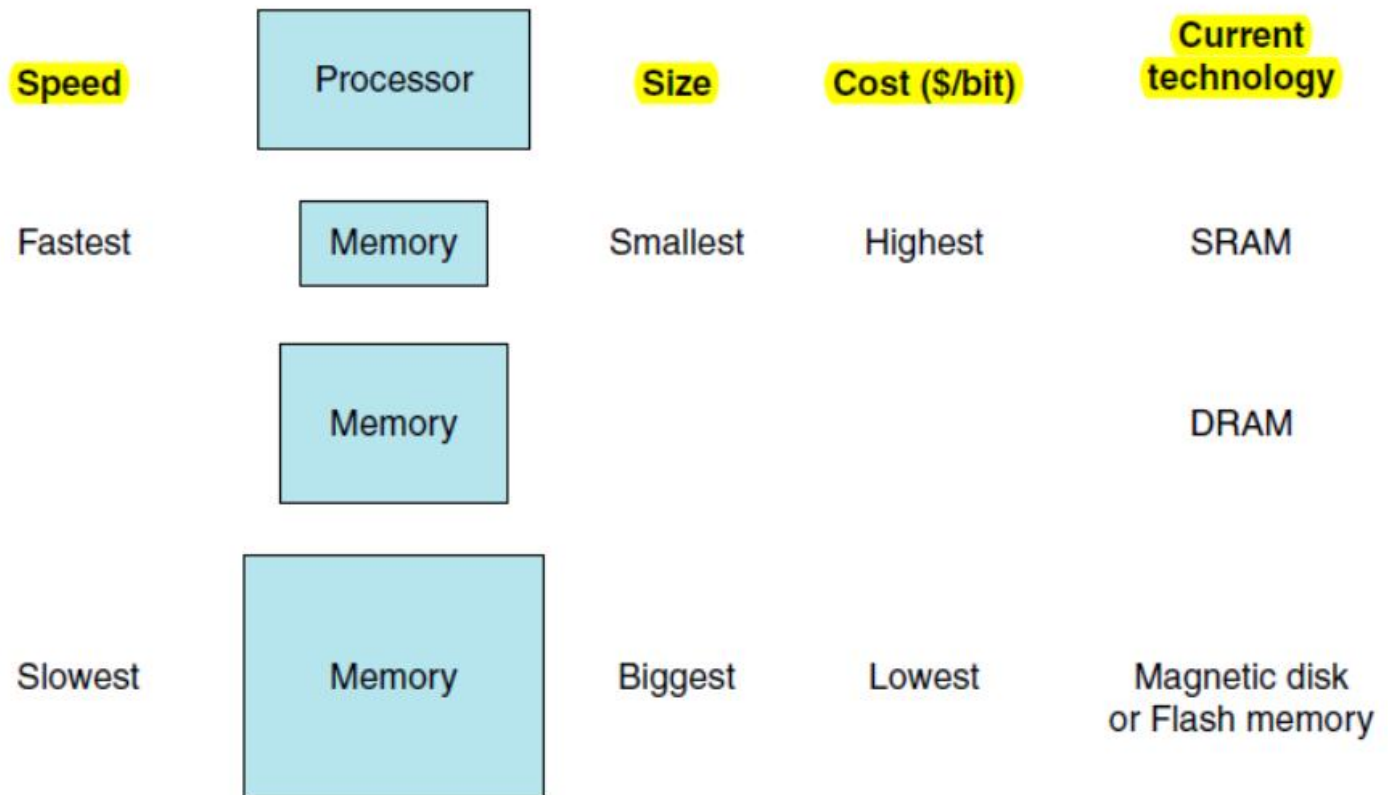
- From the earliest days of computing, programmers have wanted unlimited amounts of fast memory.
- The topics in this chapter aid programmers by creating that illusion
- Analogy of research in library

Principle of Locality

- Programs access a small proportion of their address space at any time
- Temporal locality
 - Items accessed recently are likely to be accessed again soon
 - e.g., instructions in a loop, induction variables
- Spatial locality
 - Example

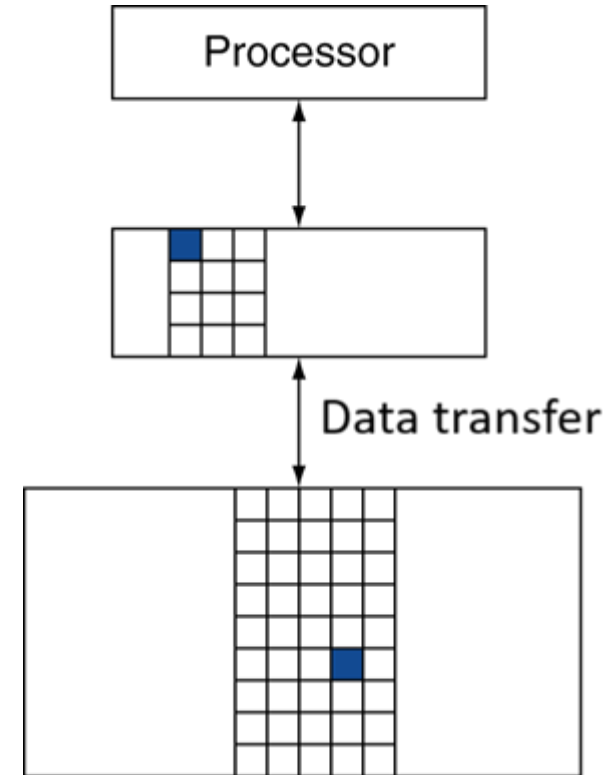
Taking Advantage of Locality

- Memory hierarchy (tradeoff)
- Goal: Maximize use of cheapest memory with access speeds of fastest memory!



Memory Hierarchy Levels

- Block (aka line):
 - Minimum unit of information present at a level / copied from one level to another
- If accessed data is present in upper level
 - Hit: access satisfied by upper level
 - Hit time
 - Hit rate
- If accessed data is absent
 - Miss: block copied from lower level
 - Miss time/Penalty
 - Miss rate



Memory importance

- Concepts used to build memory systems affect many other aspects of a computer, including
 - how applications use the computer
 - how the operating system manages memory
 - how compilers generate code
- Memory is a major factor in determining performance of a program execution on a processor
- Programmers need to understand and exploit the fact that memory is a hierarchy to get good performance
- Similarly, computer designers develop sophisticated mechanisms for improving the memory system performance.
 - Some of these mechanisms we will study in this chapter

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Memory Technologies

- There are four primary technologies used today in memory hierarchies.
 1. Dynamic RAM (DRAM)
 - Relatively cheaper but slower
 - *Main memory (RAM)* is made using this technology
 2. Static RAM (SRAM)
 - Relatively expensive but faster
 - *Caches* are made using this technology
 3. Magnetic disk
 - Cheapest but slowest
 - *Hard disks* are made using this technology
 4. Flash Disk
 - Often used as portable storage devices e.g *USBs*, SD cards etc.

Memory Technologies

Memory technology	Typical access time	\$ per GiB in 2020
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$3–\$6
Flash semiconductor memory	5,000–50,000 ns	\$0.06–\$0.12
Magnetic disk	5,000,000–20,000,000 ns	\$0.01–\$0.02

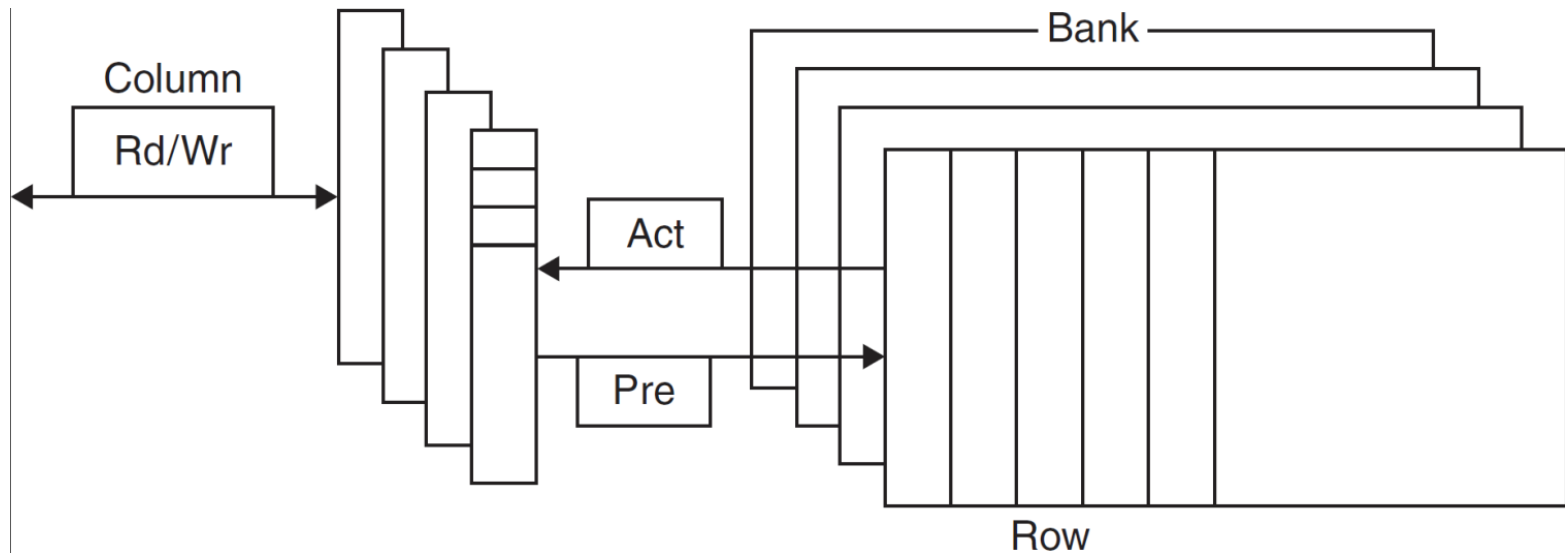
The next few slides will discuss in detail these technologies

SRAM Technology

- SRAMs are integrated circuits working as memory arrays
 - 6-8 transistors latched together to store a bit
- Single access port that can provide either a read or a write
- Fixed access time to any datum, though read and write times can differ
- Does not need refreshing to keep the data unlike DRAMs
- Now all levels of caches are integrated onto the processor chip (thanks to Moore's law)

DRAM Technology

- Data stored as a charge in a capacitor
 - Charge leakages over time and hence must periodically be refreshed, hence the term *dynamic*
- Use a single transistor per bit and hence **denser and cheaper** than SRAMs



DRAM Technology

Moore's
Law at
play here!

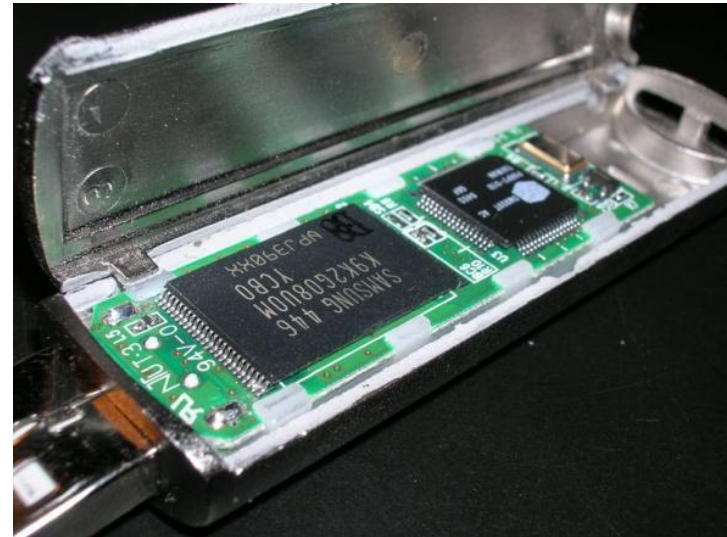
Year introduced	Chip size	\$ per GiB	Total access time to a new row/column	Average column access time to existing row
1980	64 Kibibit	\$6,480,000	250 ns	150 ns
1983	256 Kibibit	\$1,980,000	185 ns	100 ns
1985	1 Mebibit	\$720,000	135 ns	40 ns
1989	4 Mebibit	\$128,000	110 ns	40 ns
1992	16 Mebibit	\$30,000	90 ns	30 ns
1996	64 Mebibit	\$9,000	60 ns	12 ns
1998	128 Mebibit	\$900	60 ns	10 ns
2000	256 Mebibit	\$840	55 ns	7 ns
2004	512 Mebibit	\$150	50 ns	5 ns
2007	1 Gibibit	\$40	45 ns	1.25 ns
2010	2 Gibibit	\$13	40 ns	1 ns
2012	4 Gibibit	\$5	35 ns	0.8 ns
2015	8 Gibibit	\$7	30 ns	0.6 ns
2018	16 Gibibit	\$6	25 ns	0.4 ns

Advanced DRAM Features

- Row buffer
 - Allows several words to be read and refreshed in parallel -> Burst mode
- Synchronous DRAM
 - Allows for data transfers at both rising and falling edges of the clock (Double data-rate (DDR))
 - Improves bandwidth
- Dual-inline-Memory Modules (DIMM)
 - For servers which require multiple DRAMs (typically 4-16)
 - Improves bandwidth

Flash Storage

- **Nonvolatile** semiconductor storage
 - 100× – 1000× faster than (hard) disk
 - Smaller, lower power, more robust
 - **But more \$/GB**

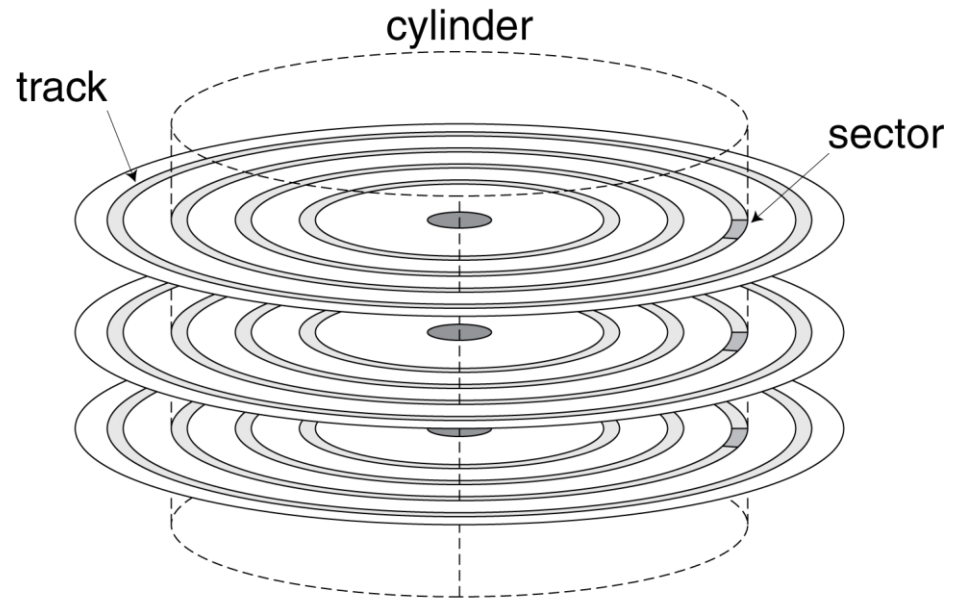


Flash Types

- NOR flash: bit cell like a NOR gate
 - Random read/write access
 - Used for instruction memory in embedded systems
- NAND flash: bit cell like a NAND gate
 - Denser (bits/area), but block-at-a-time access
 - Cheaper per GB
 - Used for USB keys, media storage, ...
- Flash bits wears out after 1000's of accesses
 - Not suitable for direct RAM or disk replacement
 - Wear leveling: remap data to less used blocks

(Hard) Disk Storage

- Nonvolatile, rotating magnetic storage



Disk Sectors and Access

- Each sector records
 - Sector ID
 - Data (512 bytes, 4096 bytes proposed)
 - Error correcting code (ECC)
 - Used to hide defects and recording errors
 - Synchronization fields and gaps
- Access to a sector involves
 - Queuing delay if other accesses are pending
 - Seek: move the heads
 - Rotational latency
 - Data transfer
 - Controller overhead

Disk Access Example

- Given
 - 512B sector, 15,000 rpm, 4ms average seek time, 100MB/s transfer rate, 0.2ms controller overhead, idle disk
- Average read time
 - 4ms seek time
 - + $\frac{1}{2} / (15,000/60) = 2\text{ms}$ rotational latency
 - + $512 / 100\text{MB/s} = 0.005\text{ms}$ transfer time
 - + 0.2ms controller delay
 - = 6.2ms
- If actual average seek time is 1ms
 - Average read time = 3.2ms

Disk Comparison

- Two primary differences between magnetic disks and semiconductor memory technologies (RAM, Flash)
- Magnetic disks are nonvolatile like flash, but unlike flash there is no write wear-out problem.
- However, flash is much more rugged and hence a better match for personal mobile devices.

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Cache Memory

- Cache memory
 - The level of the memory hierarchy closest to the CPU
- Assume if the cache contains recent data accesses $X_1, \dots, X_{n-1}$, and the processor requests X_n

X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_3

a. Before the reference to X_n

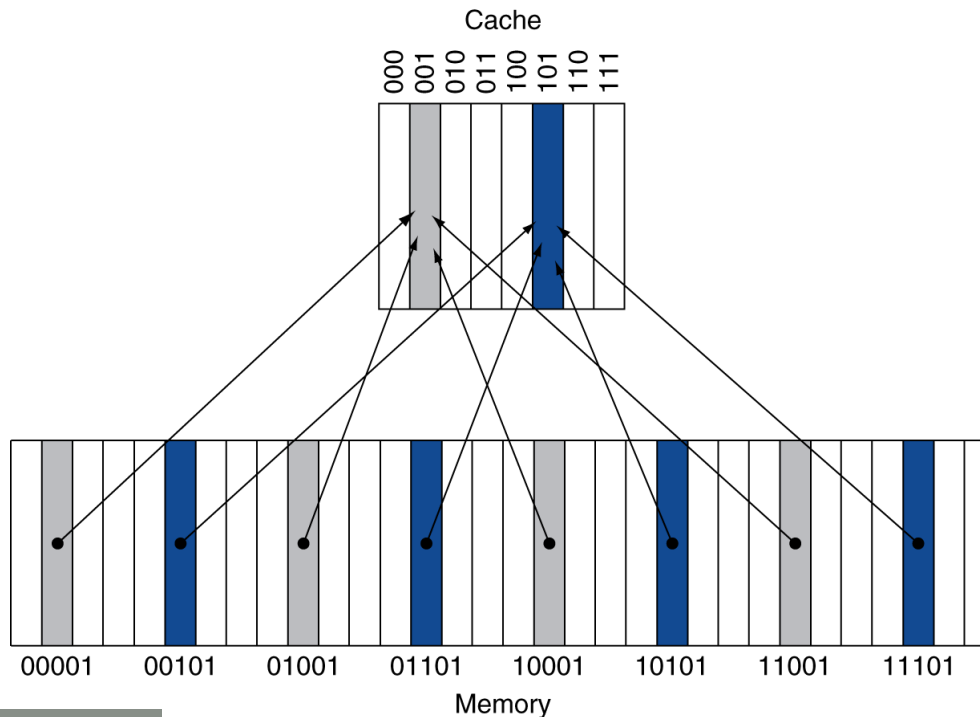
X_4
X_1
X_{n-2}
X_{n-1}
X_2
X_n
X_3

b. After the reference to X_n

- How do we know if the data is present?
- Where do we look?

Direct Mapped Cache

- A good strategy would be to fix the cache location of data based on its memory address
- Direct mapped: only one choice
 - (Block address) modulo (#Blocks in cache)



- If #Blocks is a power of 2, one can use low-order $\log_2(\text{\#Blocks})$ address bits

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
 - Store the high-order bits from block-address as well
 - Called the tag
- What if there is no valid data in a location?
 - Valid bit: 1 = present, 0 = not present
 - Initially 0

Cache Example

- Consider an 8-blocks, 1 word/block, direct mapped cache
- Below is a sequence of nine memory references to the initially empty cache

Decimal address of reference	Binary address of reference
22	10110 _{two}
26	11010 _{two}
22	10110 _{two}
26	11010 _{two}
16	10000 _{two}
3	00011 _{two}
16	10000 _{two}
18	10010 _{two}
16	10000 _{two}